

National Tsing Hua University  
Department of Electrical Engineering  
EE429200 IC Design Laboratory, Fall 2025

**Homework Assignment #1 (9%, total 100 points)**  
**Three-input Raster Operations (ROP3) for Computer Graphics**  
Assigned on Sep. 18, 2025  
Due by **Oct 2, 2025**

## Introduction

Raster operation (ROP) provide fundamental bit-level functions for computer graphics, which we are using all the time on the PC and mobile phone. Among them, ROP3 functions are designed for three 8-bit inputs, which contains pattern P [7:0], source S [7:0], and background D [7:0], respectively. They consist of 256 different functions which are specified by an 8-bit input, mode [7:0]. Table I shows examples of fifteen such ROP3 functions.

| Pattern (P)    | 1 1 1 1 0 0 0 0 | Mode (Hex) | Function Name | Boolean Equation |
|----------------|-----------------|------------|---------------|------------------|
| Source (S)     | 1 1 0 0 1 1 0 0 |            |               |                  |
| Background (D) | 1 0 1 0 1 0 1 0 |            |               |                  |
| <b>Result</b>  | 0 0 0 0 0 0 0 0 | 8'h00      | BLACKNESS     | 0                |
|                | 0 0 0 1 0 0 0 1 | 8'h11      | NOTSRCERASE   | $\sim(D S)$      |
|                | 0 0 1 1 0 0 1 1 | 8'h33      | NOTSRCCOPY    | $\sim S$         |
|                | 0 1 0 0 0 1 0 0 | 8'h44      | SRCERASE      | $S \& \sim D$    |
|                | 0 1 0 1 0 1 0 1 | 8'h55      | DSTINVERT     | $\sim D$         |
|                | 0 1 0 1 1 0 1 0 | 8'h5A      | PATINVERT     | $D \wedge P$     |
|                | 0 1 1 0 0 1 1 0 | 8'h66      | SRCINVERT     | $D \wedge S$     |
|                | 1 0 0 0 1 0 0 0 | 8'h88      | SRCAND        | $D \& S$         |
|                | 1 0 1 1 1 0 1 1 | 8'hBB      | MERGEPAINT    | $D   \sim S$     |
|                | 1 1 0 0 0 0 0 0 | 8'hC0      | MERGECOPY     | $P \& S$         |
|                | 1 1 0 0 1 1 0 0 | 8'hCC      | SRCCOPY       | $S$              |
|                | 1 1 1 0 1 1 1 0 | 8'hEE      | SRCPAINT      | $D   S$          |
|                | 1 1 1 1 0 0 0 0 | 8'hF0      | PARTCOPY      | $P$              |
|                | 1 1 1 1 1 0 1 1 | 8'hFB      | PATPAINT      | $D   P   \sim S$ |
|                | 1 1 1 1 1 1 1 1 | 8'hFF      | WHITENESS     | 1                |

Table I: Fifteen ROP3 functions with their modes and Boolean equations. The middle columns show bit-level examples for P [7:0] = 8'hF0, S [7:0] = 8'hCC, and D [7:0] = 8'hAA.

Moreover, ROP3 works on each bit position  $i$  independently, and the formulation is given as follows:

```

temp1 [7:0] = 8'h1 << {P[i], S[i], D[i]};
temp2 [7:0] = temp1 [7:0] & mode [7:0];
result [i] = | temp2 [7:0];

```

It is encouraged to verify this simple formulation with the Boolean functions in Table 1. Note that if the 256 functions are implemented in brute-force ways and selected by a large multiplexer, it could consume much more hardware resource.

We will verify this statement after introducing synthesis tools in Homework 3, please look forward to it.

In this assignment, you need to design this ROP3 module in Verilog RTL using different implementation methods as shown in Figure 1. This module includes input ‘**din\_valid**’ that represents the input data **din**, **sel** and **mode** are valid. To decide the value of **P**, **S**, **D**. The 4\*N-bits input data need to be split into 4 N-bits data, selected and arranged with **sel**, notice that the selection and arrangement of input data can refer Table II.

$$W = Din [4 * N - 1 : 3 * N]$$

$$X = Din [3 * N - 1 : 2 * N]$$

$$Y = Din [2 * N - 1 : 1 * N]$$

$$Z = Din [1 * N - 1 : 0 * N]$$

After writing the RTL code, you need to test it with the testbench either provided by TA in Part 1, and written by yourself in Part2, 3. Details are as follows:

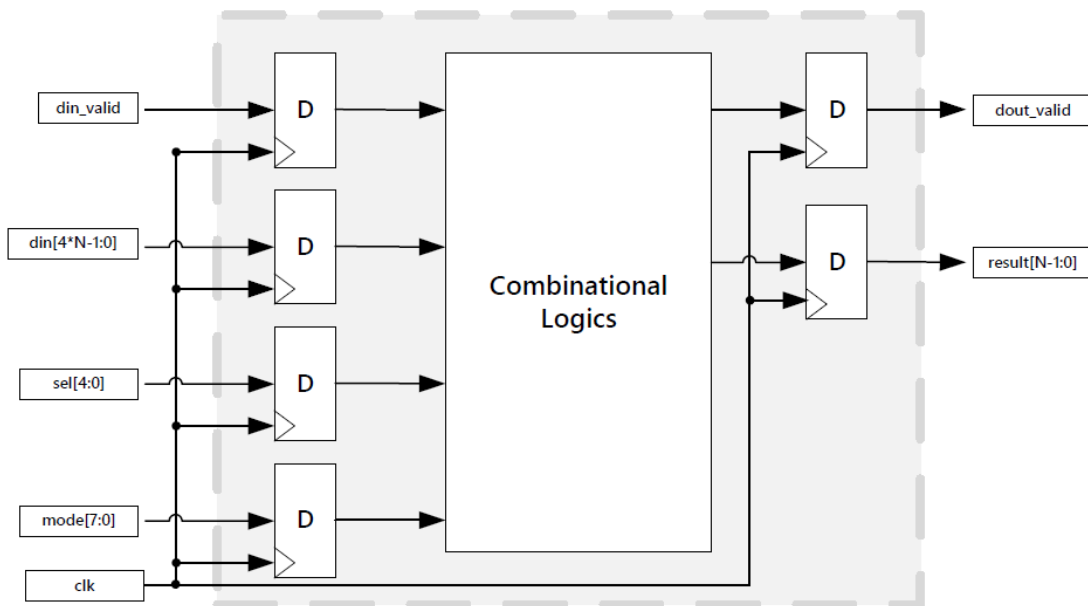


Figure 1: Module rop3 block diagram view.

| sel[4:0] | P | S | D |
|----------|---|---|---|
| 5'd0     | W | X | Y |
| 5'd1     | W | Y | X |
| 5'd2     | X | W | Y |
| 5'd3     | Y | W | X |
| 5'd4     | X | Y | W |
| 5'd5     | Y | X | W |
| 5'd6     | W | X | Z |
| 5'd7     | W | Z | X |
| 5'd8     | X | W | Z |
| 5'd9     | Z | W | X |
| 5'd10    | X | Z | W |
| 5'd11    | Z | X | W |
| 5'd12    | W | Y | Z |
| 5'd13    | W | Z | Y |
| 5'd14    | Y | W | Z |
| 5'd15    | Z | W | Y |
| 5'd16    | Y | Z | W |
| 5'd17    | Z | Y | W |
| 5'd18    | X | Y | Z |
| 5'd19    | X | Z | Y |
| 5'd20    | Y | X | Z |
| 5'd21    | Z | X | Y |
| 5'd22    | Y | Z | X |
| 5'd23    | Z | Y | X |
| Others   | 0 | 0 | 0 |

Table II: Selection of input data for P, S and D.

**Part 1 (20 points)**

For the first part, design a Verilog module (\HW1\hdl\rop3\_lut16.v) that implements those fifteen functions listed in Table I. Note that this module should be implemented using multiplexer (i.e. look-up table). For those modes which do not appear in Table I, set the computation result to zero. Note that din\_valid and dout\_valid represents input and output data valid respectively. After finishing the RTL, test it with the testbench provided by TA:

1. Complete \HW1\hdl\rop3\_lut16.v
2. Change directory to \HW1\sim\part1\
3. Run \$>> vcs -f test\_rop3\_lut16.f -full64 -R -debug\_access+all +v2k +define+N=8

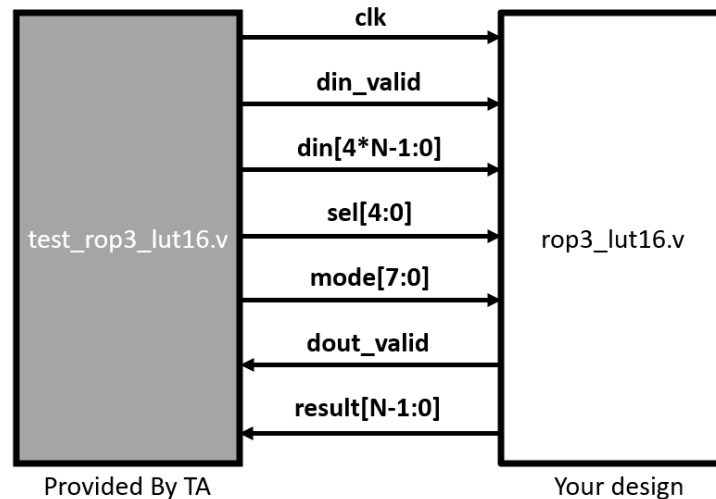


Figure 2. Part 1 block diagram.

## Part 2 (30 points)

In this part, you need to implement the ROP3 function using the magical formulation mentioned above. After finishing the RTL (`\HW1\hdl\rop3_smart.v`), you need to test it by comparing its behavior with the RTL you wrote in Part 1(`rop3_lut16.v`).

1. Complete `\HW1\hdl\rop3_smart.v`
2. Change directory to `\HW1\sim\part2\`, write a testbench `test_rop3_smart.v` to test your RTL. Note that:
  - (a) This testbench should send identical inputs to both modules (`rop3_lut16` & `rop3_smart`), and compare their computation results. A reference testing architecture is shown in Figure 3.
  - (b) This testbench should generate all the modes listed in Table I and all possible combinations of P, S and D. Try to use for-loop to generate these input stimuluses.
3. Write a `README.txt` to describe how you organize your testbench to test the RTL.
4. You can dump the waveform for debugging in the testbench, but remember to delete them (`*.fsdb` and statements in testbench) when submitting.

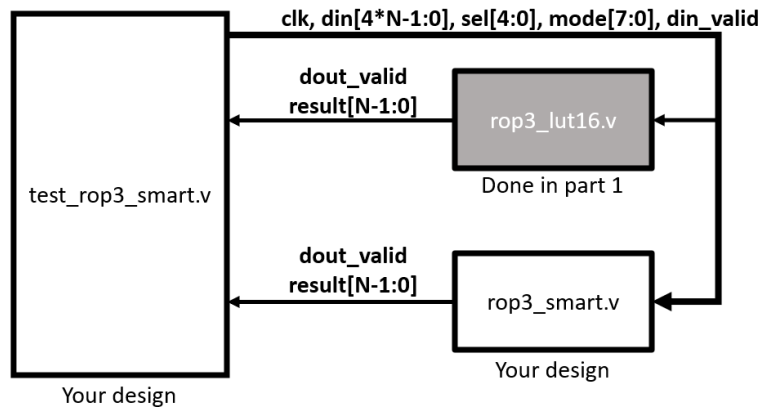


Figure 3. Part 2 block diagram.

### Part 3 (40 points)

There are actually 256 possible functions for ROP3, but we only list 15 of them in Table I. Find out all the 256 Boolean functions for ROP3, and use a large multiplexer to implement it. After finishing the RTL, try to compare the results with `rop3_smart.v` you implemented in Part2. (Note: The overall simulation may take more than ten minutes. You may use `$display` to inform some internal status, e.g. display a message whenever a new function mode is tested.)

(Hint: Try to observe Table 1 carefully, the bit example of {P, S, D} we show in Table 1 is meaningful)

1. Complete `\HW1\hdl\rop3_lut256.v`
2. Change directory to `\HW1\sim\part3\`, write a testbench `test_rop3.v` to test your RTL. Note that:
  - (a) This testbench should send identical inputs to both modules(`rop3_smart` and `rop3_lut256`), and compare their computation results. A reference testing architecture is shown in Figure 4.
  - (b) This testbench should generate all of the 256 modes and all possible combinations of P, S and D. Try to use for-loop to generate these input stimuluses.
3. Write a `README.txt` to describe how you organize your testbench to test the RTL.
4. You can dump the waveform for debugging in the testbench, but remember to delete them (`*.fsdb` and statements in testbench) when submitting.

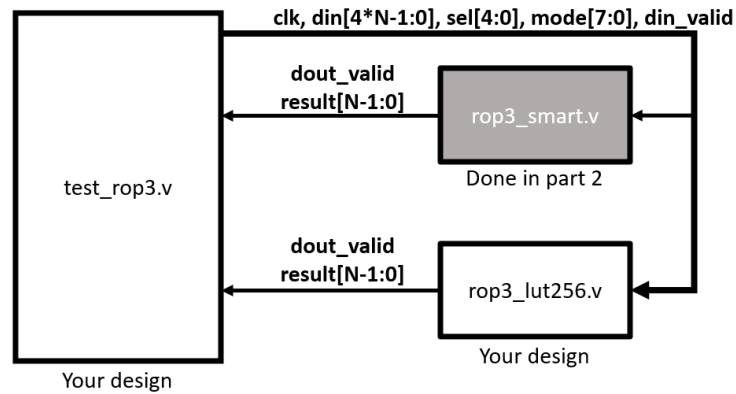


Figure 3. Part 3 block diagram.

### RTL I/O name definition

| I/O name   | Number of bits | Description                                      |
|------------|----------------|--------------------------------------------------|
| clk        | 1              | Clock input                                      |
| din_valid  | 1              | Input valid. 0: input non-valid, 1: input valid. |
| din        | 4N             | Data input                                       |
| sel        | 5              | Selection and arrangement input                  |
| mode       | 8              | Function mode input                              |
| dout_valid | 1              | Result valid. 0: data non-valid, 1: data valid.  |
| result     | N              | Result output                                    |

### Note

1. The word length N should be **parameterizable** in both RTL and testbench files. You should test them with at least two settings, e.g. 4 and 8, before delivering your homework. We will run simulation with different settings to test your codes.
2. All RTLs should be **synthesizable**. i.e. rop3\_lut16, rop3\_lut256 and rop3\_smart should pass Spyglass examination before submitting.
3. **Using generative AI is not recommended.** If used, please explain in the report how it was applied

### Deliverable

1. Synthesizable Verilog **rop3\_lut16.v**, **rop3\_lut256.v** and **rop3\_smart.v**.
2. Testbench **test\_rop3\_smart.v** (Part 2) and **test\_rop3.v** (Part 3).
3. Spyglass pass reports.
4. Text files **README.txt** to describe how you organize the testbench in Part 2, Part 3. And how you find out all of the 256 functions.
5. Create a directory named **HW1\_{studentID}**. For example, if your student ID is u123456789, the directory following the specified organization. Finally, compress the directory into a zip file named **HW1\_{studentID}.zip**.

**Note: 10 points punishments for wrong file delivery.**

| Directory             | Filename                 | Description                         |
|-----------------------|--------------------------|-------------------------------------|
| HW1_{sID}/hdl         | rop3_lut16.v             | Part 1 module                       |
|                       | rop3_smart.v             | Part 2 module                       |
|                       | rop3_lut256.v            | Part 3 module                       |
| HW1_{sID}/rpt         | spyglass_rop3_lut16.rpt  | Spyglass report for part 1 module   |
|                       | spyglass_rop3_smart.rpt  | Spyglass report for part 2 module   |
|                       | spyglass_rop3_lut256.rpt | Spyglass report for part 3 module   |
| HW1_{sID}/sim/part_2/ | test_rop3_smart.v        | Part 2 testbench                    |
|                       | test_rop3_smart.f        | Part 2 RTL pre-simulation file list |
|                       | README.txt               | Description file                    |
| HW1_{sID}/sim/part_3/ | test_rop3.v              | Part 3 testbench                    |
|                       | test_rop3.f              | Part 3 pre-simulation file list     |
|                       | README.txt               | Description file                    |

## File Organization

| Directory      | Filename              | Description                         |
|----------------|-----------------------|-------------------------------------|
| HW1/hdl/       | rop3_lut16.v          | Part 1 module                       |
|                | rop3_smart.v          | Part 2 module                       |
|                | rop3_lut256.v         | Part 3 module                       |
| HW1/pat/data/  | rop3_lut16_golden.dat | Part 1 pre-simulation golden data   |
|                | rop3_lut16_input.dat  | Part 1 pre-simulation input data    |
| HW1/sim/part_1 | test_rop3_lut16.f     | Part 1 RTL pre-simulation file list |
|                | test_rop3_lut16.v     | Part 1 testbench                    |
| HW1/sim/part_2 | test_rop3_smart.v     | Part 2 testbench                    |
| HW1/sim/part_3 | test_rop3_lut256.v    | Part 3 testbench                    |
| HW1/util/      | spyglass_ref.tcl      | Spyglass reference script           |
|                | spyglass_tutorial.pdf | Tutorial material for spyglass      |